CS 690S AI Alignment — Homework 3

RLHF on CartPole

Sagnik Chatterjee

October 21, 2026

1 Setup and results

The pipeline follows the three parts of the assignment:

1. **Synthetic demonstrations.** A reward-to-go vanilla policy gradient (`vpg.py`) was trained on `CartPole-v0` for 10 epochs, checkpointing after each epoch. These 10 deliberately under-trained policies are the "demonstrators".

2. **Preference-based reward learning.** One trajectory was rolled out from each checkpoint, and 20 random pairs of those trajectories were labelled by *ground-truth* return (the higher-return trajectory is preferred). A two-hidden-layer MLP reward network $r_\theta : \mathbb{R}^4 \to \mathbb{R}$ (`utils.py`, `Net`) scores each state; the predicted return of a trajectory is $\hat{J}_\theta(\tau) = \sum_{s_t \in \tau} r_\theta(s_t)$. The two predicted returns of a pair are used as the logits of a two-way `CrossEntropyLoss`, i.e. the Bradley–Terry model

$$P_\theta(\tau_i \succ \tau_j) \;=\; \frac{\exp \hat{J}_\theta(\tau_i)}{\exp \hat{J}_\theta(\tau_i) + \exp \hat{J}_\theta(\tau_j)},$$

   trained for 100 epochs with Adam (lr $= 10^{-3}$) and saved to `reward.params`.

3. **RL on the learned reward.** The same policy-gradient algorithm was rerun for 50 epochs, using *only* r_θ as the reward signal (the ground-truth reward is never seen by the learner), checkpointing into `rlhf/`.

Measured performance (5 rollouts each, ground-truth return on `CartPole-v0`, where 200 is the maximum achievable):

Policy	Average ground-truth return
Best demonstrator (`synthetic/policy_checkpoint9`)	69.2
RLHF policy (`rlhf/policy_checkpoint49`)	**163.0**

Averaging over 20 rollouts instead of 5, the ten demonstrator checkpoints score roughly 25–60, and the single best trajectory ever produced by any demonstrator was around 120–140 steps. The learning curve of the RLHF policy over all 50 checkpoints is shown in Figure 1.

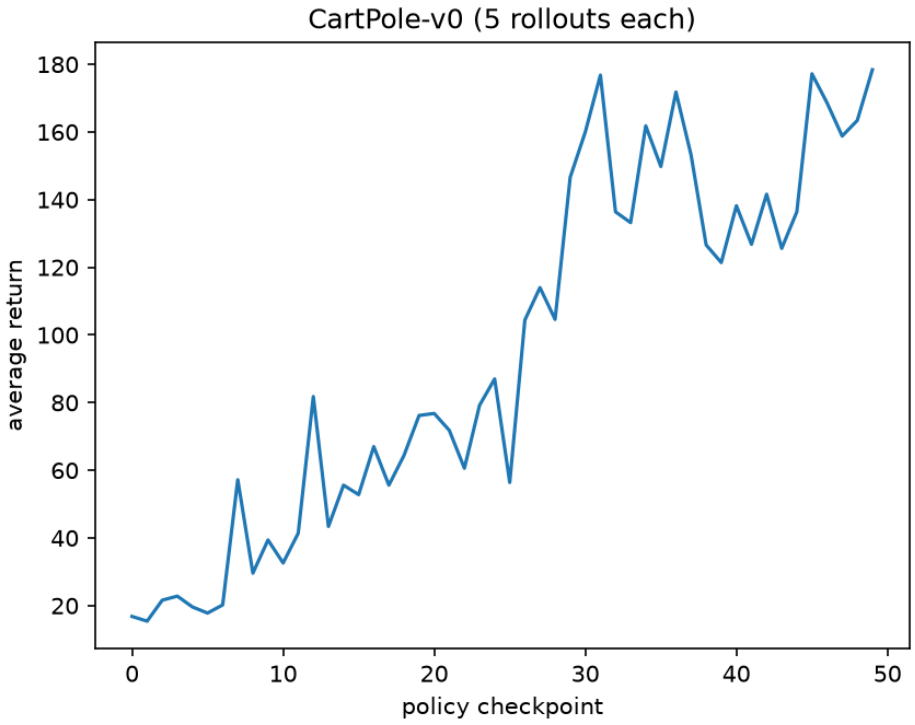


Figure 1: Average ground-truth return (5 rollouts per checkpoint) of the policy trained with RL on the *learned* reward function, across all 50 checkpoints, evaluated on `CartPole-v0` (200-step cap, so 200 is optimal). Returns climb from roughly 15 at checkpoint 0 to a band of 120–180 over the last 20 checkpoints, peaking around 178. Note that the policy being plotted never observed the ground-truth reward during training. The variance between neighbouring checkpoints is expected from a plain policy gradient with only 5 evaluation rollouts per point.

2 Questions

(1) Was the RLHF method able to learn a better policy than the best policy used to collect training data?

Yes, and by a wide margin. The best demonstrator (checkpoint 9 of the synthetic run) averages 69.2 over 5 rollouts (≈ 44 over 20 rollouts), while the final RLHF policy averages 163.0 over the same protocol — roughly a 2.4× improvement. The RLHF policy also beats the best *single trajectory* in the preference dataset: no demonstration exceeded ~ 140 steps, whereas the best RLHF checkpoints average 175–178 out of a maximum of 200 (Figure 1) and individual rollouts hit the 200-step cap, i.e. optimal play. So the learner extrapolated beyond every piece of behaviour it was shown.

(2) Why is it possible to learn a policy much better than the best trajectory in the training data?

Because what is learned from the preferences is a *reward function*, not a policy, and the reward function generalises outside the support of the demonstrations.

- **Preferences are relative, and the ranking carries directional information.** A label "$\tau_i \succ \tau_j$" does not say "imitate τ_i"; it says "move in the direction that separates τ_i from

τ_j". Pairs drawn from checkpoints of different quality tell the reward network which states distinguish better behaviour from worse (pole near upright, cart near centre, small angular velocity). Extrapolating that trend *past* the best demonstration is the natural continuation of the gradient it encodes — the same argument as in T-REX (Brown et al., *Extrapolating Beyond Suboptimal Demonstrations via Inverse RL from Observations*).

- **The reward decomposes over states, so returns extrapolate over time.** Since $\hat{J}_\theta(\tau) = \sum_t r_\theta(s_t)$ with r_θ positive on "good" states, a policy that stays in good states *longer* than any demonstrator automatically gets a higher learned return. The reward model never caps out at the best observed return; longer balancing keeps accumulating reward, so the RL optimiser is actively rewarded for surpassing the data. Credit assignment also improves: a per-state reward is a much denser, better-shaped signal than a single trajectory-level score.

- **RL supplies its own fresh experience.** The policy-gradient learner interacts with the real environment and generates states the demonstrators never visited. It is not limited to the training distribution the way a purely offline imitation method is; it only needs the reward model to rank those new states sensibly.

The flip side is that this is extrapolation, and it is only safe while the learned reward stays accurate off-distribution. If the policy drifts far enough from the demonstration support, the reward model can be exploited (reward hacking) — CartPole is simple and low-dimensional enough (4-D state, obvious structure) that the extrapolation holds here.

(3) How is RLHF different from Inverse RL or behavioral cloning?

Behavioral cloning (BC) is supervised learning on state–action pairs: it fits $\pi_\phi(a \mid s)$ to match the demonstrator by maximising $\log \pi_\phi(a_t \mid s_t)$ over the dataset. There is no reward, no environment interaction, and no notion of "better than". Its explicit target is to *reproduce* the demonstrator, so the demonstrator's performance is a ceiling. It also suffers from covariate shift: small errors push the agent into states absent from the data, where it has no supervision (the DAgger problem).

Classical Inverse RL does recover a reward function, but under the assumption that the demonstrations are (near-)optimal — typically by matching feature expectations or maximum entropy. The learned reward is the one that *rationalises the demonstrator*, so a policy that optimises it converges back to roughly demonstrator-level behaviour. It is also typically expensive: many formulations require solving the forward RL problem in an inner loop.

RLHF differs on three axes:

- *Supervision signal.* Comparisons/rankings between behaviours, not action labels and not an optimality assumption. Humans are far better at judging "which of these two is better" than at demonstrating optimal control or writing down a reward.

- *What is learned.* A reward model (Bradley–Terry over trajectory returns), which is then optimised by a standard RL algorithm. The policy is the product of *optimisation*, not of imitation.

- *Ceiling.* Because the data is explicitly labelled with *relative* quality and can come from deliberately suboptimal behaviour, RLHF can rank and then exceed everything it was shown. BC and standard IRL are both anchored to the demonstrator.

In this assignment a ground-truth-return oracle stands in for the human labeller, which is exactly the standard synthetic-preference setup: it isolates the algorithmic question from human-annotation noise. A practical RLHF system would also need to handle noisy, inconsistent, and sometimes mistaken human labels.

(4) What would have happened if Inverse RL or behavioral cloning were run on the same trajectories used for RLHF?

Both would have produced a policy that is *at best* as good as the demonstrations, i.e. an average return somewhere in the 20–60 range, and plausibly worse.

- **Behavioral cloning** would have been trained on an undifferentiated mixture of all ten checkpoints — including checkpoint 0, which barely balances the pole for ~ 25 steps. BC has no way to tell good data from bad: every state–action pair is treated as a target to match. The result would be a policy near the *average* of the demonstrators, roughly 30–40 return, which is worse than just copying the best checkpoint. Compounding error under covariate shift would likely drag it lower still. Note also that the trajectories stored here are *observation-only* (`generate_rollout` records states, not actions), so plain BC could not even be run on this dataset without re-collecting actions.
- **Inverse RL** would recover a reward function that explains this mixture as if it were optimal behaviour. Since the mixture contains many early failures (pole falling after 20–30 steps), the inferred reward would treat "drop the pole fairly quickly" as rational, and optimising it would reproduce mediocre behaviour rather than surpass it. The demonstrator distribution becomes the target, so there is nothing to extrapolate along.

The key difference is the preference labels. The same ten trajectories carry a strong learning signal once they are *ranked*: the ranking reveals which direction in behaviour space is "better", and that direction can be followed beyond the data. Treated as things to copy, the same trajectories are a mediocre dataset and bound the learner to mediocre performance.